

Problem Solving Using C (25VBT0C103)



www.onlinejain.com

Module: 01

Programming Foundations & C Basics

Module Information

Field	Details
Course Title	Problem Solving using C
Course Code	25VBT0C103
Module Number	Module 1 of 5
Module Title	Programming Foundations and C Basics
Semester	I
Credits	4

Module Overview

Every piece of software you interact with daily — the UPI app on your smartphone, the ATM that dispenses cash, the traffic management system at a busy intersection, the weather alert on your television — was built by a programmer who started exactly where you are now: learning the fundamentals of a programming language.

Module 1 lays the essential groundwork for this journey using the C programming language. C is not just a language to learn — it is the language that built the modern computing world. The Linux operating system, the Python interpreter, the MySQL database engine, and even the firmware inside the microcontrollers of ISRO's satellites are written in C. Mastering C gives you a direct view into how computers actually work.

This module begins by placing programming in its real-world context — understanding why software is needed and where C fits in modern systems. You will then progressively build your vocabulary in C: variables and constants (the building blocks of data), data types (how data is classified and stored), operators (the tools to manipulate data), and the essential input/output functions that let your program communicate with the user.

The module then moves into the logic and decision-making capabilities of C: conditional statements (if, if-else, switch) that let your program respond to different situations, and looping constructs (for, while, do-while) that let your program repeat actions efficiently. Real-time scenarios — electricity bill calculation, ATM PIN verification, mobile recharge, and traffic signal simulation — are woven throughout to make every concept immediately applicable.

The module closes with best practices in code documentation and commenting — the professional discipline that separates readable, maintainable code from confusing, fragile code. By the end of Module 1, you will be able to write complete, functional C programs that read input, make decisions, and produce meaningful output.

Learning Objectives

By the end of this module, students will be able to:

- Recall the history and significance of C in systems and embedded programming and identify its role in modern software development.
- Identify and describe variables, constants, keywords, and data types used in C programs.
- Recall the purpose and syntax of operators, input/output functions, and expression evaluation in C.
- Understand and describe the behaviour of conditional statements (if, if-else, switch) in program logic.
- Understand and explain the operation of looping constructs (for, while, do-while) and loop control statements (break, continue, goto).

Learning Outcomes

Upon successful completion of this module, students will be able to:

- Identify the correct data type for a given real-world variable and justify the choice based on memory and range.
- Write and execute simple C programs that accept user input and produce formatted output using printf and scanf.
- Construct programs using if, if-else, and switch statements to model real-world decision scenarios such as utility bill grading and ATM transactions.
- Develop loop-based programs using for, while, and do-while constructs to solve repetitive tasks such as recharge cycle processing and report generation.
- Apply proper code commenting and documentation practices to write readable and maintainable C programs.

Table of Topics

Sub-Unit	Topic	Coverage
1.1	Role of Programming and Introduction to C	Why programming matters, history of C, C in real-world systems
1.2	Structure of a C Program	Header files, main(), compilation flow, first program
1.3	Variables, Constants, and Keywords	Naming rules, const, #define, reserved words
1.4	Data Types	int, float, double, char, short, long, sizeof()
1.5	Operators and Expressions	Arithmetic, relational, logical, bitwise, assignment, ternary
1.6	Input and Output in C	printf, scanf, getchar, putchar, gets, puts, format specifiers
1.7	Conditional Statements	if, if-else, nested if-else, switch-case, real-world decisions
1.8	Looping Constructs	for, while, do-while, nested loops, iteration scenarios
1.9	Loop Control: break, continue, goto	Early exit, skipping iterations, jump statements
1.10	Code Documentation and Commenting	Single-line, multi-line comments, documentation standards

1.1 Role of Programming and Introduction to C

Imagine you walk up to an ATM and enter your PIN. Within milliseconds, the machine checks your credentials, communicates with a bank server, verifies your account balance, and either dispenses cash or declines your request — all without human involvement. Behind this seamless interaction is a set of carefully written instructions called a program, executed by a computer.

Programming is the process of writing such instructions in a language that a computer can understand and execute. In a world driven by digital transactions, smart devices, and automated systems, programming is the foundational skill that makes all of it possible.

1.1.1 Why C?

C is one of the oldest and most influential programming languages, designed by Dennis Ritchie at Bell Labs (AT&T) in 1972. Despite being over 50 years old, C remains one of the most widely used languages in the world — and for very good reasons:

- **Closeness to Hardware:** C gives programmers direct access to memory through pointers and allows low-level operations. This makes it ideal for writing operating systems, device drivers, and embedded firmware.
- **Portability:** A C program written on one machine can be compiled and run on virtually any other machine with minimal changes. This made C the universal language for system software.
- **Performance:** C programs execute very fast because they compile directly to machine code with little overhead. This is critical in real-time systems where milliseconds matter.
- **Foundation for Other Languages:** C++ is an extension of C. Java, Python, and PHP are implemented in C. Understanding C gives you insight into how all these languages work internally.
-

1.1.2 Where Is C Used Today?

Domain	Real-World Indian Example
Operating Systems	Linux kernel (used in Android phones — 600 million+ in India) is written in C.
Embedded Systems	ISRO's PSLV onboard flight software, Aadhaar biometric scanner firmware, two-wheeler ABS controllers.
Banking & Payments	NPCI's UPI transaction routing engine, ATM software (Diebold, NCR — deployed at SBI, HDFC).
Telecom	Base station controllers for Jio 5G, Airtel 4G — written in C/C++ for real-time signal processing.
Databases	MySQL, PostgreSQL, SQLite — all database engines used by Indian startups are implemented in C.

💡 Did You Know?

Dennis Ritchie created C to rewrite the Unix operating system, which was originally written in assembly language. The resulting Unix + C combination was so powerful that it spread across universities worldwide — including IIT Bombay, which was among the first Indian institutions to adopt Unix in the early 1980s. This laid the foundation for India's computer science education ecosystem.

1.2 Structure of a C Program

Every C program, regardless of its size or complexity, follows a standard structure. Understanding this structure is like understanding the layout of a standard business letter — once you know where each part goes, writing becomes natural.

1.2.1 The Basic Structure

Consider this first program — a simple welcome message for a railway booking counter:

```
/* Program: Railway Counter Welcome Message */  
  
/* Author : BCA Student, Semester I      */  
  
#include <stdio.h>      /* Standard Input/Output library */  
  
int main() {  
    printf("Welcome to Indian Railways Booking Counter!\n");  
    printf("Please have your Aadhaar card ready.\n");  
    return 0;  
}
```

1.2.2 Breaking Down Each Part

- Comments (`/* ... */`): Explanatory text ignored by the compiler. Used to describe the program's purpose — a professional practice.
- `#include <stdio.h>`: A preprocessor directive. It tells the compiler to include the Standard Input/Output header file, which provides `printf()` and `scanf()`. Think of it as importing a toolbox.

- `int main()`: Every C program starts execution from the `main()` function. The `int` before it means the function will return an integer value to the operating system.
- `{` and `}`: Curly braces define the body of the function — all statements inside are part of `main()`.
- `printf(...)`: A function call that prints text to the screen. `\n` is an escape sequence for a new line.
- `return 0`: Signals to the operating system that the program ended successfully.
0 = success; non-zero = error.

1.2.3 Compilation and Execution Flow

Understanding how a C program goes from text to execution is essential:

Step	Stage	What Happens
1	Source Code (.c)	You write the program in a text editor and save as filename.c
2	Preprocessor	<code>#include</code> and <code>#define</code> directives are processed; macros expanded.
3	Compiler	Source code converted to object code (.o or .obj) — machine-readable but not yet executable.
4	Linker	Library functions (<code>printf</code> , <code>scanf</code>) are linked to your object code.
5	Executable (.exe / a.out)	Final runnable program. On Linux: <code>gcc program.c -o program</code> then <code>./program</code>

1.3 Variables, Constants, and Keywords

In any real-world system, information changes over time. The balance in your bank account changes with every transaction. Your mobile data usage changes every minute you stream a video. In programming, variables are the containers that hold such changing information during program execution.

1.3.1 Variables

A variable is a named storage location in memory that holds a value which can change during program execution. Think of it as a labelled box: the label is the variable name, and the contents are the value.

Scenario: Consider a mobile prepaid recharge system. We need to track:

- customer's current balance — changes with recharges and calls
- recharge amount — entered by the user
- remaining days of validity — decreases daily

```
int validity_days = 28;    /* Variable: changes daily */
```

```
float current_balance = 0.0; /* Variable: changes with usage */
```

```
float recharge_amount;    /* Declared but not yet assigned */
```

Rules for Naming Variables

- Must begin with a letter (a–z, A–Z) or underscore (_). Cannot start with a digit.
- Can contain letters, digits (0–9), and underscores. No spaces or special characters.
- Case-sensitive: balance, Balance, and BALANCE are three different variables.
- Cannot use C keywords (int, float, if, for, etc.) as variable names.

Valid Variable Names	Invalid Variable Names
account_balance, pinCode, upild, temp1	2ndUser (starts with digit), my balance (space), float (keyword)

1.3.2 Constants

A constant is a value that does not change during program execution. For example, the GST rate on mobile recharge (currently 18%), the number of hours in a day (24), or the speed of light — these are fixed values that should not be accidentally changed.

There are two ways to define constants in C:

Using #define (Preprocessor Constant)

```
#define GST_RATE 0.18 /* 18% GST on telecom services */  
  
#define MAX_PIN_ATTEMPTS 3 /* Maximum wrong PIN tries at ATM */  
  
#define BANK_NAME "State Bank of India"
```

Using the const Keyword

```
const float PI = 3.14159;  
  
const int WORKING_HOURS = 8; /* Fixed working hours per shift */  
  
const int OTP_LENGTH = 6; /* UPI OTP is always 6 digits */
```

Difference: #define is a text substitution done before compilation (preprocessor). const is a typed, compiler-checked constant — preferred in modern C programming for better type safety.

1.3.3 Keywords

Keywords are reserved words that have a specific meaning in C. They cannot be used as variable names or function names. C has 32 standard keywords:

Data Types	Control Flow	Storage Class	Others
int, float, double, char, void, short, long, unsigned, signed	if, else, switch, case, default, for, while, do, break, continue, return, goto	auto, extern, static, register	sizeof, typedef, struct, union, enum, const, volatile

🔍 Analysing Misconceptions: Variable Names and Case Sensitivity

✗ MYTH: C is not case-sensitive, so "Balance" and "balance" refer to the same variable.

✓ REALITY: C is fully case-sensitive. "Balance", "balance", and "BALANCE" are three entirely different variables stored at different memory locations. This is a common source of bugs for beginners. Consistent naming conventions (e.g., all lowercase with underscores: `account_balance`) help avoid this confusion.

1.4 Data Types

Every piece of data in a computer program has a type — it tells the compiler what kind of value the variable holds, how much memory to allocate, and what operations are valid. Choosing the right data type is like choosing the right container: you would not store water in a paper bag or rocks in a glass vial.

1.4.1 Fundamental Data Types in C

Data Type	Size	Range	Real-World Use	Example
char	1 byte	-128 to 127	Store a grade ('A', 'B'), menu choice	char grade = 'A';
int	4 bytes	-2.1B to +2.1B	Account number, OTP, age, count	int otp = 482916;
short	2 bytes	-32768 to 32767	Port numbers, small counters	short port = 8080;
long	8 bytes	Very large integers	Aadhaar number (12 digits), PAN transactions	long aadhaar = 987654321012L;
float	4 bytes	$\pm 3.4 \times 10^{38}$ (6 decimals)	Temperature, petrol price, percentage	float petrol_price = 102.45;

double	8 bytes	$\pm 1.7 \times 10^{308}$ (15 decimals)	Stock prices, GPS coordinates	double gps_lat = 28.613939;
--------	---------	---	-------------------------------	-----------------------------

1.4.2 Choosing the Right Data Type — Real-World Example

Scenario: You are building a billing system for an electricity distribution company (like MSEDCL or BESCO). Consider the variables needed:

/* Electricity Bill Management System - Data Type Selection */	
char consumer_category;	/* R=Residential, C=Commercial, I=Industrial */
int consumer_id;	/* Unique consumer ID: e.g., 12345678 */
long meter_reading;	/* Cumulative meter reading can be very large */
float units_consumed;	/* e.g., 245.5 kWh this month */
double bill_amount;	/* Precise billing: e.g., Rs. 1847.63 */
int due_date;	/* Day of month: 1-31 */

1.4.3 The sizeof() Operator

The sizeof() operator returns the number of bytes occupied by a data type or variable on the current platform. This is essential when writing programs for embedded systems where memory is scarce.

```
#include <stdio.h>
```

```
int main() {
```

```

printf("Size of int   : %lu bytes\n", sizeof(int));

printf("Size of float : %lu bytes\n", sizeof(float));

printf("Size of double : %lu bytes\n", sizeof(double));

printf("Size of char  : %lu bytes\n", sizeof(char));

return 0;

}

```

Output on a 64-bit system: int = 4 bytes, float = 4 bytes, double = 8 bytes, char = 1 byte.

✓ Let's Check

Q.No	Type	Question
1	MCQ	An Aadhaar number has 12 digits (max value: 999999999999). Which data type is most appropriate? (A) int (B) short (C) long (D) float
2	MCQ	Which keyword is used to define a constant that the compiler type-checks? (A) #define (B) const (C) static (D) extern
3	Fill in Blank	In C, variable names are _____ — "Total" and "total" are treated as different variables.
4	Fill in Blank	The function that returns the memory size of a data type in bytes is _____.()
5	True/False	The keyword "int" can be used as a variable name in C. (True/False)

6	True/False	A char variable in C occupies 1 byte of memory and can store a single character. (True/False)
---	------------	---

Answers:

Q.No	Answer
1	(C) long — int can hold up to ~2.1 billion (10 digits); long handles 12-digit Aadhaar numbers.
2	(B) const — unlike #define which is a text substitution, const is a typed constant checked by the compiler.
3	Case-sensitive
4	sizeof
5	False — "int" is a reserved keyword; using it as a variable name causes a compilation error.
6	True — char is 1 byte and stores ASCII characters like 'A', '0', '#'.

1.5 Operators and Expressions

Operators are the action words of C — they specify what computation or comparison to perform on values. An expression combines variables, constants, and operators to produce a result. Mastering operators is essential because every real-world computation — from calculating GST on an invoice to verifying a PIN — is built from them.

1.5.1 Arithmetic Operators

Scenario: Calculate the electricity bill for a residential consumer in Bengaluru (BESCOM rates, 2024):

Operator	Symbol	Description	Bill Example
Addition	+	Add two values	total = base_charge + fixed_charge
Subtraction	-	Subtract right from left	units = current_reading - previous_reading
Multiplication	*	Multiply two values	bill = units * rate_per_unit
Division	/	Divide (quotient)	avg_daily_units = monthly_units / 30
Modulus	%	Remainder after division	extra_units = units % 100 (excess over slab)

```
/* Electricity Bill Calculation - BESCOM Residential */

#include <stdio.h>

int main() {

    int current_reading = 3245, previous_reading = 2980;
```

```

int units_consumed = current_reading - previous_reading; /* 265 units */

float rate = 5.75; /* Rs per unit (0-250 slab) */

float fixed_charge = 50.0;

float gst_rate = 0.05; /* 5% GST on electricity */


float base_bill = units_consumed * rate;

float gst_amount = base_bill * gst_rate;

float total_bill = base_bill + fixed_charge + gst_amount;


printf("Units Consumed : %d kWh\n", units_consumed);

printf("Base Bill      : Rs. %.2f\n", base_bill);

printf("GST (5%%)      : Rs. %.2f\n", gst_amount);

printf("Total Payable : Rs. %.2f\n", total_bill);

return 0;

}

```

1.5.2 Relational Operators

Relational operators compare two values and return 1 (true) or 0 (false). Used in every decision-making scenario:

Operator	Symbol	ATM Scenario Example
Equal to	==	entered_pin == stored_pin (PIN verification)
Not equal to	!=	entered_pin != stored_pin (wrong PIN)

Greater than	>	balance > withdrawal_amount (sufficient funds)
Less than	<	attempts < 3 (still within allowed PIN tries)
Greater than or equal	>=	age >= 18 (eligible for account opening)
Less than or equal	<=	withdrawal <= 10000 (daily ATM limit check)

1.5.3 Logical Operators

Logical operators combine multiple conditions into a single true/false result:

- **&& (AND):** Both conditions must be true. Example: if (balance >= amount && account_active) — withdraw only if enough balance AND account is not blocked.
- **|| (OR):** At least one condition must be true. Example: if (user_is_admin || user_is_manager) — allow access if either role is held.
- **! (NOT):** Reverses the truth value. Example: if (!card_blocked) — proceed only if card is NOT blocked.

1.5.4 Increment, Decrement, and Assignment

```
int attempts = 0;

attempts++;    /* attempts becomes 1 — after each wrong PIN */

attempts--;    /* attempts becomes 0 — (rarely used for PIN tracking) */
```

```
float balance = 5000.0;

balance -= 1500.0; /* Equivalent to: balance = balance - 1500.0 */

balance += 200.0; /* Credit Rs. 200 (interest) */
```

1.5.5 Ternary Operator

The ternary operator is a compact if-else in one line: condition ? value_if_true : value_if_false

```
/* Jio prepaid: check if data pack is active */

int data_balance_mb = 512;

char *status = (data_balance_mb > 0) ? "Data Available" : "Recharge Required";

printf("Data Status: %s\n", status);

/* Output: Data Status: Data Available */
```

1.6 Input and Output in C

A program that cannot communicate with the outside world — reading data from users, files, or sensors, and displaying results — has very limited utility. Input/Output (I/O) is the interface between your program and the real world. In C, the `stdio.h` library provides a rich set of I/O functions.

1.6.1 `printf()` — Formatted Output

`printf()` prints formatted text to the standard output (usually the terminal). The format string can contain text and format specifiers — placeholders that get replaced by variable values.

Specifier	Data Type	Example
%d or %i	int	<code>printf("%d", 1234);</code> → 1234
%f	float / double	<code>printf("%.2f", 99.5);</code> → 99.50
%c	char	<code>printf("%c", 'A');</code> → A
%s	string (char array)	<code>printf("%s", "Jio");</code> → Jio
%ld	long int	<code>printf("%ld", 987654321012L);</code> → Aadhaar no.
%lu	unsigned long	<code>printf("%lu", sizeof(double));</code> → 8

1.6.2 scanf() — Formatted Input

scanf() reads formatted input from the keyboard. The & (address-of) operator is required before variable names — it tells scanf where in memory to store the value.

```
/* UPI Payment — Receive Transaction Details from User */

#include <stdio.h>

int main() {

    long sender_account;

    float amount;

    char upi_id[20];    /* Character array for string input */

    printf("Enter your account number : ");

    scanf("%ld", &sender_account);

    printf("Enter UPI ID      : ");

    scanf("%s", upi_id); /* No & needed for arrays */

    printf("Enter transfer amount   : Rs. ");

    scanf("%f", &amount);

    printf("\n--- Transaction Summary ---\n");

    printf("From Account : %ld\n", sender_account);

    printf("To UPI ID   : %s\n", upi_id);
```

```
printf("Amount      : Rs. %.2f\n", amount);  
  
return 0;
```

```
}
```

1.6.3 Character I/O: `getchar()`, `putchar()`, `gets()`, `puts()`

- `getchar()`: Reads a single character from keyboard. Useful for reading Yes/No confirmations (e.g., "Confirm transaction? Y/N: ").
- `putchar()`: Prints a single character to screen.
- `gets()`: Reads an entire line including spaces (deprecated in C11 due to buffer overflow risk; use `fgets()` instead).
- `puts()`: Prints a string followed by a newline.

```
/* ATM: Confirm transaction */  
  
char confirm;  
  
printf("Confirm withdrawal of Rs. 2000? (Y/N): ");  
  
confirm = getchar();  
  
if (confirm == 'Y' || confirm == 'y') {  
    puts("Transaction approved. Please collect cash.");  
} else {  
    puts("Transaction cancelled.");  
}
```

💡 Did You Know?

The `printf` function was designed so cleverly that its name is short for "print formatted". Its % format specifiers were directly borrowed by Python's print formatting system, Java's `String.format()`, and even SQL's `PRINT` statement. When Ritchie designed `printf`, he could not have imagined it would influence dozens of languages written over the next 50 years.

1.7 Conditional Statements

Real-world systems constantly make decisions: Should this ATM transaction be approved? Which electricity billing slab does this consumer fall under? Is this blood pressure reading normal or alarming? Conditional statements allow your C program to make such decisions based on the values of variables.

1.7.1 The if Statement

The simplest conditional: execute a block of code only when a condition is true.

Syntax: `if (condition) { /* statements */ }`

Real-Time Scenario: Low balance alert for a prepaid mobile subscriber (Airtel):


```
#include <stdio.h>

int main() {

    float data_balance_mb = 85.5;

    if (data_balance_mb < 100.0) {

        printf("⚠ Low Data Alert: Only %.1f MB remaining.\n", data_balance_mb);

        printf("Recharge now at airtel.in/recharge\n");

    }

    printf("Data Balance: %.1f MB\n", data_balance_mb);

    return 0;

}
```

1.7.2 The if-else Statement

Execute one block if the condition is true, and a different block if false.

Real-Time Scenario: ATM cash withdrawal — check if sufficient balance exists:

```
#include <stdio.h>

int main() {

    float account_balance = 3500.00;

    float withdrawal_amount;
```

```
printf("Enter withdrawal amount: Rs. ");
scanf("%f", &withdrawal_amount);

if (withdrawal_amount <= account_balance) {
    account_balance -= withdrawal_amount;
    printf("Transaction Successful!\n");
    printf("Dispensing Rs. %.2f\n", withdrawal_amount);
    printf("Remaining Balance: Rs. %.2f\n", account_balance);
} else {
    printf("Insufficient Balance.\n");
    printf("Available Balance: Rs. %.2f\n", account_balance);
}

return 0;
}
```

1.7.3 Nested if-else (Multi-Slab Decision)

Real-Time Scenario: BESCO electricity billing with progressive slabs (2024 rates):

```
#include <stdio.h>
```

```
int main() {
    int units;
    float bill = 0.0;
```

```
printf("Enter units consumed this month: ");  
  
scanf("%d", &units);  
  
  
if (units <= 30) {  
  
    bill = 0.0;          /* Free for BPL consumers */  
  
    printf("BPL Category: No charge for first 30 units.\n");  
  
} else if (units <= 100) {  
  
    bill = units * 3.15;  
  
} else if (units <= 200) {  
  
    bill = 100 * 3.15 + (units - 100) * 5.00;  
  
} else if (units <= 500) {  
  
    bill = 100 * 3.15 + 100 * 5.00 + (units - 200) * 6.00;  
  
} else {  
  
    bill = 100*3.15 + 100*5.00 + 300*6.00 + (units-500) * 7.00;  
  
}  
  
printf("Units: %d kWh | Bill: Rs. %.2f\n", units, bill);  
  
return 0;  
  
}
```

1.7.4 The switch Statement

switch is ideal when a variable can take one of many specific values and you want different actions for each. It is cleaner than a long chain of if-else for fixed, discrete choices.

Real-Time Scenario: Railway ticket booking — select travel class:

```
#include <stdio.h>

int main() {
    int class_choice;
    float base_fare = 250.0;
    float total_fare;

    printf("Indian Railways — Select Travel Class\n");
    printf("1. Sleeper (SL)\n");
    printf("2. AC 3-Tier (3A)\n");
    printf("3. AC 2-Tier (2A)\n");
    printf("4. AC First Class (1A)\n");
    printf("Enter choice (1-4): ");
    scanf("%d", &class_choice);

    switch (class_choice) {
        case 1:
```

<pre>total_fare = base_fare * 1.0; printf("Sleeper Class: Rs. %.2f\n", total_fare);</pre>
<pre>break;</pre>
<pre>case 2:</pre>
<pre>total_fare = base_fare * 2.5; printf("AC 3-Tier : Rs. %.2f\n", total_fare); break;</pre>
<pre>case 3: total_fare = base_fare * 3.8; printf("AC 2-Tier : Rs. %.2f\n", total_fare);</pre>
<pre>break;</pre>
<pre>case 4: total_fare = base_fare * 5.5; printf("AC First Cls : Rs. %.2f\n", total_fare); break;</pre>
<pre>default:</pre>
<pre>printf("Invalid choice. Please select 1–4.\n");</pre>
<pre>}</pre>
<pre>return 0; }</pre>

🔍 Analysing Misconceptions: == vs = in Conditions

✗ MYTH: Writing `if (balance = 0)` checks whether balance is zero.

✓ REALITY: `if (balance = 0)` is an assignment, not a comparison! It sets balance to 0 (which is false) and the if block never executes. The correct relational operator for comparison is `==` (double equals): `if (balance == 0)`. This is one of the most common bugs in C programming. Good compilers warn about this, but beginners must always double-check conditions.

✓ Let's Check

Q.No	Type	Question
1	MCQ	What is the output of: <code>int x = 7; printf("%d", (x % 2 == 0) ? 1 : 0);</code> (A) 0 (B) 1 (C) 7 (D) Compilation error
2	MCQ	In a switch statement, which keyword prevents execution from "falling through" to the next case? (A) exit (B) stop (C) return (D) break
3	Fill in Blank	In the expression <code>if (a = 5)</code> , the operator <code>=</code> performs _____, not comparison.
4	Fill in Blank	The _____ clause in a switch statement handles all values not matched by any case.
5	True/False	An if statement without an else clause still executes correctly when the condition is false — it simply skips the if block. (True/False)

6	True/False	A switch statement in C can test floating-point values like 3.14. (True/False)
---	------------	--

Answers:

Q.No	Answer
1	(A) $0 - 7 \% 2 = 1$ (not 0), so the condition is false and the ternary returns 0.
2	(D) break — without break, execution "falls through" and executes the next case's code.
3	Assignment
4	default
5	True — if the condition is false, execution continues at the statement after the closing brace.
6	False — switch only works with integer-type expressions (int, char, enum). Float/double causes a compilation error.

1.8 Looping Constructs

Repetition is at the heart of computing power. An electricity distribution company generates bills for millions of consumers each month. A bank sends OTPs to thousands of customers every minute. A traffic camera captures and analyses thousands of frames per second. All of this repetition is handled elegantly by looping constructs.

1.8.1 The for Loop

Use the for loop when you know in advance exactly how many times a block should repeat. Its three-part header makes the loop counter setup, condition, and update visible in one line.

Syntax: for (initialisation; condition; update) { /* statements */ }

Real-Time Scenario: Generate a monthly report showing daily electricity units for a 7-day period:

```
#include <stdio.h>

int main() {

    float daily_units[] = {12.5, 14.0, 11.8, 16.2, 13.5, 18.0, 15.3};

    float total = 0.0;

    int day;

    printf(" Day | Units (kWh)\n");
    printf("-----|-----\n");

    for (day = 1; day <= 7; day++) {

        printf(" %3d | %.1f\n", day, daily_units[day - 1]);
```



```
total += daily_units[day - 1];  
  
}  
  
printf("-----|-----\n");  
  
printf(" Total| %.1f kWh\n", total);  
  
return 0;  
  
}
```

1.8.2 The while Loop

Use the while loop when repetition should continue as long as a condition is true, and the exact number of iterations is not known in advance.

Syntax: while (condition) { /* statements */ }

Real-Time Scenario: ATM PIN entry — allow up to 3 attempts before blocking the card:

```
#include <stdio.h>  
  
int main() {  
  
    int correct_pin = 4782;  
  
    int entered_pin;  
  
    int attempts = 0;  
  
    int max_attempts = 3;  
  
    while (attempts < max_attempts) {  
  
        printf("Enter 4-digit PIN (Attempt %d/%d): ", attempts + 1, max_attempts);
```

```
scanf("%d", &entered_pin);

if (entered_pin == correct_pin) {

    printf("✓ PIN Accepted. Welcome!\n");

    return 0; /* Exit program on success */

} else {

    attempts++;

    if (attempts < max_attempts)

        printf("X Incorrect PIN. %d attempt(s) remaining.\n",

            max_attempts - attempts);

}

}

printf("Card BLOCKED. Contact your bank.\n");

return 0;

}
```

1.8.3 The do-while Loop

The do-while loop executes the body at least once before checking the condition. This is perfect for menu-driven applications where you always want to show the menu at least once.

Syntax: `do { /* statements */ } while (condition);`

Real-Time Scenario: Mobile recharge menu — keep showing options until user chooses to exit:

```
#include <stdio.h>

int main() {

    int choice;

    float wallet_balance = 150.00;

    do {

        printf("\n=== Jio Recharge Menu ===\n");

        printf("Wallet Balance: Rs. %.2f\n", wallet_balance);

        printf("1. Recharge Rs. 49 (1 GB/day, 28 days)\n");

        printf("2. Recharge Rs. 149 (2 GB/day, 28 days)\n");

        printf("3. Recharge Rs. 299 (2 GB/day, 56 days)\n");

        printf("4. Exit\n");

        printf("Enter choice: ");

        scanf("%d", &choice);

        switch (choice) {

            case 1: wallet_balance -= 49; printf("Recharged Rs. 49!\n"); break;

            case 2: wallet_balance -= 149; printf("Recharged Rs. 149!\n"); break;

            case 3: wallet_balance -= 299; printf("Recharged Rs. 299!\n"); break;

            case 4: printf("Thank you. Goodbye!\n"); break;

            default: printf("Invalid choice.\n");
```

```
}  
  
} while (choice != 4);
```

```
return 0;
```

```
}
```

💡 Did You Know?

Loops are so fundamental to computing that modern hardware has special support for them. The branch predictor in Intel and ARM processors (which we studied in Computer Architecture) specifically tracks loop-back branches and predicts them as "taken" with very high accuracy — because loops repeat hundreds of times before exiting. This is why loop-heavy numerical code (like matrix multiplication) runs much faster than code with many unpredictable branches.

1.9 Loop Control Statements: break, continue, goto

Sometimes a loop needs to exit early when an exceptional condition is met, or skip the current iteration without exiting. C provides three jump statements for this purpose: break, continue, and goto.

1.9.1 break — Early Exit from Loop

break immediately exits the innermost loop (or switch) and continues with the statement after it.

Real-Time Scenario: Scanning a queue at a vaccination centre — search for a patient by Aadhaar ID:

```
#include <stdio.h>

int main() {

    long aadhaar_queue[] = {987600000001L, 987600000002L,
                             987600000003L, 987600000004L};

    long search_id = 987600000003L;

    int n = 4, found = 0, i;

    for (i = 0; i < n; i++) {

        if (aadhaar_queue[i] == search_id) {

            found = 1;

            break; /* Stop scanning — patient found */

        }

    }

    if (found)

        printf("Patient found at queue position %d.\n", i + 1);

    else

        printf("Patient not in current queue.\n");

    return 0;

}
```

1.9.2 continue — Skip to Next Iteration

continue skips the rest of the current iteration's body and jumps to the next iteration of the loop.

Real-Time Scenario: Generate electricity bill report but skip months where the meter was faulty (reading = -1):

```
#include <stdio.h>

int main() {

    int monthly_units[] = {245, 310, -1, 280, 195, -1, 330};

    /* -1 indicates faulty meter reading */

    int months = 7, i;

    float total_billed = 0.0;

    printf("Month | Units | Status\n");

    for (i = 0; i < months; i++) {

        if (monthly_units[i] == -1) {

            printf(" %3d | --- | Meter Fault — Skipped\n", i + 1);

            continue; /* Skip billing for this month */

        }

        float bill = monthly_units[i] * 5.75;

        total_billed += bill;

        printf(" %3d | %4d | Rs. %.2f\n", i+1, monthly_units[i], bill);

    }
```

```
printf("Total Billed: Rs. %.2f\n", total_billed);

return 0;

}
```

1.9.3 goto — Unconditional Jump

goto transfers control to a labelled statement anywhere in the same function. While it exists in C and has rare legitimate uses (error handling in deeply nested code), it is generally discouraged because it makes programs hard to read and debug.

```
/* goto is rarely used — shown here for completeness */
```

```
int main() {

    int x = 5;

    if (x > 3)

        goto error_handler; /* Jump to label */
```

```
printf("Normal execution\n");
```

```
return 0;
```

```
error_handler:      /* Label definition */
```

```
printf("Error: x exceeds limit.\n");
```

```
return 1;
```

```
}
```

Best Practice: Use break and continue in preference to goto wherever possible. goto is acceptable only in error-handling code inside deeply nested loops where break cannot reach the outer level.

Analysing Misconceptions: break in Nested Loops

✗ MYTH: break exits ALL loops when used inside a nested loop.

✓ REALITY: break only exits the INNERMOST loop (or switch) that directly contains it. If you have a for loop nested inside a while loop, a break inside the for loop exits only the for loop — the while loop continues. To exit multiple levels of nesting, you typically need a flag variable or restructure the code. This is actually one of the motivations for goto in C — but modern style prefers a flag variable approach.

1.10 Code Documentation and Commenting

Imagine inheriting a 5,000-line C program written by a colleague who left the company six months ago — with no comments, no variable name explanations, and no documentation. This is one of the most frustrating experiences in software development, and it happens in the real world all the time. Good code documentation is a professional responsibility, not an optional luxury.

1.10.1 Types of Comments in C

Single-Line Comments (//)

Available from C99 onwards. Use for brief, inline explanations:

```
float gst_rate = 0.18;    // 18% GST applicable on telecom services

int max_pin_attempts = 3; // RBI mandate: card blocked after 3 wrong PINs

daily_balance -= withdrawal; // Deduct from available balance
```


Multi-Line Comments (`/* ... */`)

Use for block descriptions, function documentation, and explanations spanning multiple lines:

<code>/*</code>
<code>* Function : calculate_electricity_bill()</code>
<code>* Purpose : Computes monthly electricity bill per BESCOM 2024 slab rates</code>
<code>* Input : units_consumed (int) — monthly consumption in kWh</code>
<code>* Output : bill_amount (float) — total payable amount in INR</code>
<code>* Author : Student Name, BCA Sem I, 2024</code>
<code>* Reference : BESCOM Tariff Order No. ES/30/KERC/2024</code>
<code>*/</code>

1.10.2 File Header Comment

Every C source file should begin with a header comment that provides an overview of the program:

<code>/*</code>
<code>* File : atm_simulator.c</code>
<code>* Course : Problem Solving using C (25VBT0C103)</code>
<code>* Module : Module 1 — Programming Foundations</code>
<code>* Purpose : Simulates basic ATM operations: PIN verification,</code>
<code>* balance enquiry, and cash withdrawal.</code>
<code>* Author : Priya Menon</code>

* Date : 15-July-2024
* Version : 1.0
*
* Real-World Context: Based on SBI ATM transaction workflow.
*/

1.10.3 Commenting Best Practices

- Comment the WHY, not just the WHAT: If you write `daily_units *= 1.1`, add the comment `// Apply 10% surge pricing (peak hours 6–10 PM)` rather than `// multiply by 1.1`.
- Keep comments current: An outdated comment that contradicts the code is worse than no comment at all. Update comments whenever you change the code they describe.
- Avoid over-commenting obvious code: `int i = 0; // Set i to zero is unnecessary and adds noise.` Comment only what adds insight.
- Use meaningful variable names alongside comments: `float emi = loan_amount * monthly_rate / (1 - pow(1 + monthly_rate, -tenure));` — the variable names already tell a story; a comment on the formula source is valuable.
- Comment complex logic blocks: Any algorithm or business rule that is not immediately obvious deserves an explanation.

```
/* GOOD: Comments explain the business rule */
```

```
/* BESCOM slab: First 100 units at Rs. 3.15, next 100 at Rs. 5.00 */
```

```
if (units <= 100)
```

```
    bill = units * 3.15;
```

```
/* BAD: Restates what the code already says */
```

```
/* Multiply units by 3.15 */
```

```
bill = units * 3.15;
```

Case Study

■ Case Study: Building a Smart Electricity Billing System for MSEDCL (Maharashtra State Electricity Distribution Co. Ltd.)

Background

MSEDCL distributes electricity to over 26 million consumers across Maharashtra (excluding Mumbai). Prior to 2010, bill generation was largely manual — meter readers would visit each home, note the reading in a register, and submit it to district offices where clerks would calculate bills using printed rate charts. This process was error-prone, delayed by 10–20 days, and frequently resulted in consumer disputes. In 2010, MSEDCL contracted TCS (Tata Consultancy Services) to build an automated billing system in C and later C++.

Problem Statement

Design a console-based C module that accepts meter readings, categorises the consumer, applies the correct slab rate using conditional logic, and generates a formatted bill. The module must handle: (1) multiple consumer categories (BPL, residential, commercial, industrial), (2) progressive slab billing, (3) surcharges and taxes, and (4) validation of input data.

C Programming Concepts Applied

- Data Types: long for consumer ID, int for meter readings, float for bill amounts, char for category code.
- Operators: Arithmetic to compute units consumed (`current_reading - previous_reading`), multiplication for slab billing, modulus for excess units.
- Conditional Statements: switch to select consumer category, nested if-else for progressive slab rates, if-else for surcharge thresholds.

- Loops: for loop to process batch of 100 consumers per district in one run. do-while for re-entry on invalid input.
- I/O: scanf() to read meter data, printf() with %.2f format specifiers for precise currency output.

Sample Output

```
=====
MSEDCL CONSUMER BILL — August 2024
=====
Consumer ID   : 1234567890
Category      : Residential
Units Consumed: 340 kWh
Base Charge   : Rs. 1,705.00
Fixed Charge   : Rs.   75.00
GST (5%)      : Rs.   85.25
-----
TOTAL PAYABLE : Rs. 1,865.25
Due Date      : 25-Aug-2024
=====
```

Outcome and Learning

The TCS-built system reduced bill generation time from 10–20 days to same-day processing. Consumer disputes dropped by 62% in the first year. The system processed 26 million bills monthly using 4-level nested if-else for slab calculation, switch for category selection, and formatted printf output — all core concepts from Module 1. It ran on AIX Unix servers (IBM) using C-compiled binaries, demonstrating C's real-world reliability at scale.

Reference: MSEDCL Annual Report 2011–12; TCS Case Study — Utility Billing Automation, tcs.com/resources; Maharashtra Electricity Regulatory Commission (MERC) Tariff Order 2024.

Summary

Module 1 has introduced you to the complete foundation of C programming — from understanding why C matters in real-world systems to writing programs that make decisions and process repetitive tasks. Here is a concise review of what you have learned:

The C Language and Program Structure:

- C was created by Dennis Ritchie in 1972 and remains foundational to operating systems, embedded systems, banking software, and telecom infrastructure worldwide — including India's UPI payment network, Aadhaar infrastructure, and ISRO's embedded systems.
- Every C program has a defined structure: preprocessor directives, the `main()` function, and a return statement. The compilation pipeline goes from source code through preprocessing, compilation, linking to a final executable.

Data Fundamentals:

- Variables are named memory locations that hold changing values. Constants (`const` and `#define`) hold fixed values that should not change.
- Choosing the correct data type (`char`, `int`, `long`, `float`, `double`) determines memory usage and the range of values that can be stored. `sizeof()` helps verify memory allocation on specific platforms.
- 32 C keywords are reserved and cannot be used as variable names. Variable names are case-sensitive.

Operators and I/O:

- Arithmetic operators (+, -, *, /, %) perform computations. Relational operators (==, !=, >, <, >=, <=) compare values. Logical operators (&&, ||, !) combine conditions.
- printf() with format specifiers (%d, %f, %c, %s) produces formatted output. scanf() with % reads typed input. getchar/putchar handle single characters; gets/puts handle strings.

Decision Making and Looping:

- if, if-else, and nested if-else enable multi-level conditional logic such as electricity billing slabs and ATM balance checks.
- switch provides clean multi-branch selection for discrete values like travel class or menu options.
- for loops are ideal for fixed-count iterations (weekly reports, batch processing). while loops continue based on a condition (PIN attempts). do-while ensures at least one execution (menu-driven programs).
- break exits the innermost loop early; continue skips to the next iteration; goto provides unconditional jumps (use sparingly).

Code Quality:

- Good comments explain WHY code does what it does, not just WHAT it does. File headers, function documentation, and inline comments make code maintainable by your future self and your teammates.

Glossary

Term	Definition
Variable	A named memory location that stores a value which can change during program execution.
Constant	A named value that does not change during execution. Defined using <code>const</code> keyword or <code>#define</code> preprocessor directive.
Keyword	A reserved word in C with a predefined meaning (e.g., <code>int</code> , <code>if</code> , <code>while</code>). Cannot be used as a variable name.
Data Type	A classification that specifies the kind of value a variable holds (<code>int</code> , <code>float</code> , <code>char</code> , etc.), its memory size, and valid range.
Format Specifier	A placeholder in <code>printf/scanf</code> (e.g., <code>%d</code> , <code>%f</code> , <code>%s</code>) that specifies the type and format of data to be printed or read.
Operator	A symbol that performs an operation on one or more operands. Categories: arithmetic, relational, logical, assignment, bitwise, ternary.
Conditional Statement	A statement that executes different code blocks based on whether a condition is true or false. (<code>if</code> , <code>if-else</code> , <code>switch</code>)
Loop	A control structure that repeats a block of code. C provides three types: <code>for</code> (count-controlled), <code>while</code> (condition-controlled), <code>do-while</code> (executes at least once).
break	A jump statement that exits the innermost loop or switch block immediately, transferring control to the next statement after the loop.
Compilation	The process of translating C source code (.c file) into an executable program through preprocessing, compiling, and linking stages.

Self Assessment Questions

Section A: Multiple Choice Questions (MCQ)

Choose the most appropriate answer for each question.

1. C was developed at Bell Labs by:
 - A) Bjarne Stroustrup
 - B) Dennis Ritchie
 - C) James Gosling
 - D) Guido van Rossum
2. Which of the following is a valid variable name in C?
 - A) 2balance
 - B) my balance
 - C) account_balance
 - D) float
3. What is the size of a double data type on a 64-bit system?
 - A) 2 bytes
 - B) 4 bytes
 - C) 8 bytes
 - D) 16 bytes
4. An Aadhaar number has 12 digits. Which is the MOST suitable data type to store it?
 - A) int
 - B) short
 - C) long
 - D) float

5. What is the output of: `printf("%d", 17 % 5);`
- A) 3
 - B) 2
 - C) 5
 - D) 0
6. Which format specifier is used to print a float value rounded to 2 decimal places?
- A) %f
 - B) %d
 - C) %.2f
 - D) %2f
7. In the statement `scanf("%f", &amount);` the `&` operator means:
- A) Bitwise AND of amount
 - B) Address of the variable amount in memory
 - C) Logical AND condition
 - D) Reference to a global variable
8. Which of the following correctly defines a constant for the value of pi?
- A) `int PI = 3.14;`
 - B) `constant float PI = 3.14;`
 - C) `#define PI 3.14159`
 - D) `float PI = 3.14159;`
9. An ATM allows 3 wrong PIN attempts. Which loop is MOST appropriate for this?
- A) for loop
 - B) while loop
 - C) do-while loop
 - D) switch statement
-

10. A mobile recharge app must display the menu at least once before checking exit.

Which loop suits best?

- A) for loop
- B) while loop
- C) do-while loop
- D) if-else statement

11. What will happen if break is omitted from a switch case?

- A) Compilation error
- B) The switch exits immediately
- C) Execution falls through to the next case
- D) The default case always runs

12. Which statement skips the remaining body of the current loop iteration and proceeds to the next?

- A) break
- B) continue
- C) goto
- D) return

13. What does the following expression evaluate to: $(10 > 5) \ \&\& \ (3 < 1)$

- A) 1 (true)
- B) 0 (false)
- C) 10
- D) Compilation error

14. What does the sizeof() operator return for sizeof(char)?

- A) 0
- B) 1
- C) 2
- D) 4

-
15. Which escape sequence is used in printf to print a new line?
- A) \t
 - B) \r
 - C) \n
 - D) \0
16. The switch statement in C works with which type of expression?
- A) float only
 - B) double only
 - C) integer-type expressions (int, char)
 - D) All data types
17. Which of the following is the correct relational operator to check equality in C?
- A) =
 - B) :=
 - C) ==
 - D) ===
18. In a nested if-else for electricity billing slabs, what happens if units_consumed = 150 and the first condition checks units <= 100?
- A) The first if block executes
 - B) The first else-if checking units <= 200 is evaluated
 - C) The program terminates
 - D) A runtime error occurs
19. Which C standard introduced the // single-line comment?
- A) C89
 - B) C90
 - C) C99
 - D) C11
-

20. What is the value of x after: `int x = 10; x += 5; x *= 2;`

- A) 25
- B) 30
- C) 20
- D) 15

Section B: True / False

State whether each statement is TRUE or FALSE.

1. In C, the keywords `if` and `IF` are treated identically because C is not case-sensitive.
2. The `do-while` loop always executes its body at least once, regardless of the condition.
3. Using `#define` to define a constant is safer than using `const` because it is handled by the preprocessor.
4. The `break` statement inside a nested `for` loop exits only the innermost `for` loop.
5. `scanf()` requires the `&` (address-of) operator before variable names for all basic data types (`int`, `float`, `char`, `double`).

Section C: Short Answer Questions

Answer each question in 3–5 sentences or with a short program.

1. Write a C program that accepts the current and previous electricity meter readings, computes units consumed, and prints the bill at Rs. 6.50 per unit with 5% GST.
2. Explain the difference between the while loop and the do-while loop with a real-world scenario for each.
3. Write a C program using switch that accepts a day number (1–7) and prints the corresponding day name (Monday to Sunday). Print "Invalid" for any other input.
4. What is the difference between = (assignment) and == (equality) operators in C? Give an example of a bug that arises from confusing them.
5. Write a C program that prints numbers from 1 to 20, skipping multiples of 3, using a for loop with the continue statement. (Real-world analogy: skip faulty sensor readings.)

Self Assessment: Answer Keys

MCQ

Q.No	Correct Answer & Explanation
1	B) Dennis Ritchie — created C in 1972 at Bell Labs to rewrite the Unix operating system.
2	C) <code>account_balance</code> — starts with a letter, contains only letters and underscore. <code>"2balance"</code> starts with digit; <code>"my balance"</code> has space; <code>"float"</code> is a keyword.
3	C) 8 bytes — double is 64-bit (8 bytes) on all standard 64-bit systems, providing 15–16 significant decimal digits of precision.
4	C) long — Aadhaar has up to 12 digits (max $\sim 10^{12}$). int holds up to $\sim 2.1 \times 10^9$ (10 digits). long (8 bytes) handles values up to $\sim 9.2 \times 10^{18}$.
5	B) 2 — $17 \div 5 = 3$ remainder 2. The % operator returns the remainder after integer division.
6	C) <code>%.2f</code> — the .2 precision specifier limits the output to 2 decimal places. <code>%f</code> alone prints 6 decimal places by default.
7	B) Address of the variable amount in memory — <code>scanf</code> needs to know WHERE in memory to store the value it reads; <code>&</code> provides the memory address.
8	C) <code>#define PI 3.14159</code> — syntactically correct preprocessor constant. Option D uses float but const keyword is missing; A uses int for a float value; B uses an invalid keyword.

9	B) while loop — the number of iterations (attempts) is not fixed; the loop continues while attempts < 3 and exits when PIN is correct or attempts are exhausted.
10	C) do-while loop — the menu must display at least once before the user can choose to exit; do-while guarantees at least one execution.
11	C) Execution falls through to the next case — without break, C continues executing the next case's code until it finds a break or reaches the end of the switch. This is "fall-through" behaviour.
12	B) continue — skips remaining statements in the current iteration and jumps to the loop's next iteration (check/update step). break would exit the loop entirely.
13	B) 0 (false) — && requires BOTH conditions to be true. (10 > 5) is true, but (3 < 1) is false. true && false = false = 0.
14	B) 1 — char is defined in the C standard as exactly 1 byte. This is guaranteed on all platforms.
15	C) \n — newline escape sequence. \t is tab, \r is carriage return, \0 is null terminator.
16	C) integer-type expressions (int, char) — switch evaluates an integer expression. float and double are not permitted; the compiler will report an error.
17	C) == — single = is assignment; == is the equality comparison operator. C does not use := (Pascal/ALGOL) or === (JavaScript).
18	B) The first else-if checking units <= 200 is evaluated — 150 > 100, so the first condition is false. The next condition (150 <= 200) is true, so that block executes.

19	C) C99 — the <code>//</code> style comment was adopted from C++ into the C99 standard. Earlier C89/C90 only supported <code>/* */</code> block comments.
20	B) 30 — <code>x</code> starts at 10. <code>x += 5</code> makes <code>x = 15</code> . <code>x *= 2</code> makes <code>x = 30</code> . Operations execute left to right.

True / False Answer Key

Q.No	Answer / Explanation
1	FALSE — C is fully case-sensitive. "if" is a valid keyword; "IF" is simply an undeclared identifier and would cause a compilation error.
2	TRUE — The do-while loop evaluates the condition AFTER executing the loop body. Even if the condition is initially false, the body runs once. This is the key difference from the while loop.
3	FALSE — <code>const</code> is safer. <code>#define</code> is a text substitution with no type checking; the compiler cannot catch type mismatches. <code>const</code> is typed, scoped, and fully debuggable. Modern C guidelines (MISRA C, C99+) prefer <code>const</code> over <code>#define</code> for constants.
4	TRUE — <code>break</code> only breaks out of its immediately enclosing loop or switch. In a for loop inside a while loop, <code>break</code> exits the for loop; the while loop continues.
5	TRUE — <code>scanf()</code> needs the address of the variable to write the read value into memory. The <code>&</code> operator provides this address. Exception: arrays and char pointers (like <code>char name[]</code>) do not need <code>&</code> because array names are already pointers to their first element.

Short Answer Key

Q.No	Model Answer
1	<pre>#include <stdio.h> int main() { int cur, prev, units; printf("Current reading: "); scanf("%d", &cur); printf("Previous reading: "); scanf("%d", &prev); units = cur - prev; float bill = units * 6.50; float gst = bill * 0.05; printf("Units: %d Bill: Rs.%.2f GST: Rs.%.2f Total: Rs.%.2f\n", units, bill, gst, bill+gst); return 0; }</pre>
2	while loop: condition is checked BEFORE the body executes; if false initially, body never runs. Real-world: ATM PIN check — if card is already blocked (attempts=3), do not show PIN prompt at all. do-while loop: body executes ONCE, THEN condition is checked. Real-world: Mobile recharge menu — always show the menu at least once, then ask if the user wants to continue.
3	<pre>#include <stdio.h> int main() { int day; printf("Enter day (1-7): "); scanf("%d", &day); switch(day) { case 1: printf("Monday\n"); break; case 2: printf("Tuesday\n"); break; case 3: printf("Wednesday\n"); break; case 4: printf("Thursday\n"); break; case 5: printf("Friday\n"); break; case 6: printf("Saturday\n"); break; case 7: printf("Sunday\n"); break; default: printf("Invalid\n"); } return 0; }</pre>
4	= is the assignment operator: it copies the right-hand value into the left-hand variable. == is the equality operator: it compares two values and returns 1 (true) or 0 (false). Bug example: <code>if (balance = 0) { printf("Zero balance"); }</code> — This SETS balance to 0 (false), so the block never executes regardless of the actual balance. The correct code is: <code>if (balance == 0)</code>. Compilers often warn about this with: "warning: suggest parentheses around assignment".

5

```
#include <stdio.h> int main() { int i; for (i = 1; i <= 20; i++) { if (i % 3 == 0) continue; /* Skip multiples of 3 — like skipping faulty sensor readings */ printf("%d ", i); } printf("\n"); return 0; } /* Output: 1 2 4 5 7 8 10 11 13 14 16 17 19 20 */
```

ONLINE DEGREE PROGRAMS OFFERED

B.Com | BBA | MBA | MCA | M.Com | MA

